

H.261 与 MPEG:I / P / B 帧

上一讲我们学会了怎么在两帧之间"找位移"(运动补偿)。这一讲把镜头拉远:一整段视频到底是怎么被切成 I / P / B 三类帧、又怎么被压到只剩几十分之一的?这是老师标注的重点。

直觉类比

想象你要给朋友描述一段监控录像。第一帧你只能老老实实完整地画一遍(这就是 **I** 帧)。接下来几帧背景没怎么变,你只需说"和上一张比,那辆车往右挪了两格"(这就是 **P** 帧,只记差异)。更聪明的是:有些帧夹在两张已知帧中间,你可以同时参考前后两张来描述,记得更省(这就是 **B** 帧)。视频压缩的全部秘密,就是尽量少画"完整帧",多记"差异"。

1. 核心定义

I 帧 (Intra-frame, 帧内编码帧)

独立编码,不参考任何其他帧。只做空间冗余去除(DCT → 量化 → Zigzag → 熵编码),本质就是一张 JPEG。它是随机访问和纠错的锚点,通常每秒插入若干张。

别名:关键帧。压缩比最低($\approx 7:1$),但可独立解码。

P 帧 (Predictive-frame, 前向预测帧)

用前向预测从前一个已编码的 I 帧或 P 帧预测而来,同时去除空间冗余和时间冗余。每个宏块分配一个运动向量,编码"预测误差"。注意:P 帧里的宏块未必都用运动补偿——见下文第 3 节。

B 帧 (Bidirectional-frame, 双向预测帧)

MPEG 引入的第三种帧,支持双向运动补偿:每个宏块最多两个运动向量(一个来自前向、一个来自后向参考帧)。两个方向都匹配时,取两个匹配宏块的平均。压缩比最高($\approx 50:1$),但 B 帧本身不被别人参考。

H.261

最早的数字视频压缩标准(1990 年成为国际标准),为 ISDN 上的可视电话/视频会议设计,码率 $p \times 64$ kbps(故又称 $p \times 64$)。只有 I 帧和 P 帧两种。它确立的"基于运动补偿的帧间压缩"原理被后续所有标准继承。

MPEG-1

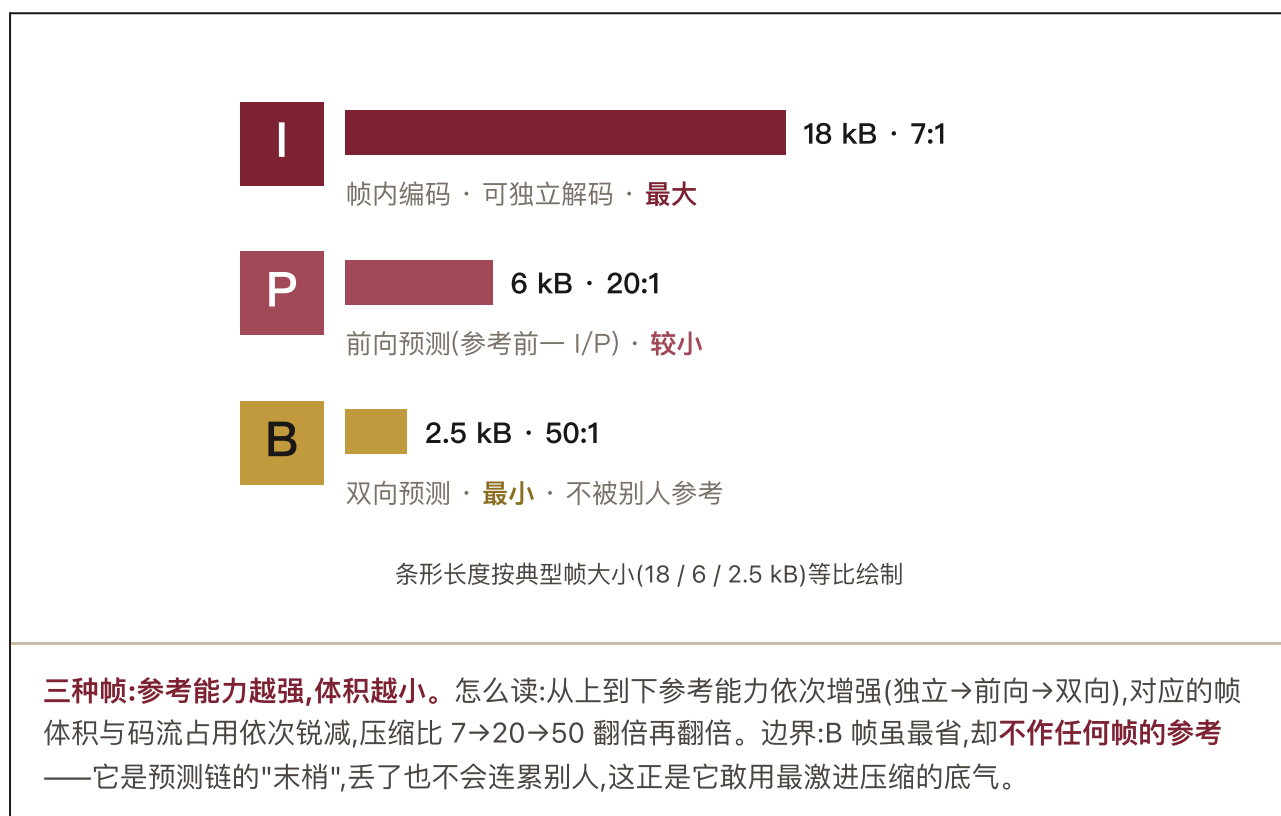
1991 年批准,面向 CD/VCD 数字存储,码率约 1.5 Mbps。采用 SIF 格式、4:2:0 色度二次采样、仅非隔行。相对 H.261 的最大区别就是新增了 B 帧和双向运动补偿。

2. I / P / B 三帧对照(必背)

这张表几乎覆盖了本讲所有辨析题和填空题。压缩比与典型大小按 MPEG-1 课件口径:

帧类型	参考方向	去除的冗余	典型大小	压缩比
I 帧	不参考任何帧(独立)	仅空间冗余	18 kB	7:1
P 帧	前向:参考前一个 I 或 P 帧	空间 + 时间冗余	6 kB	20:1
B 帧	双向:同时参考前、后的 I/P 帧	空间 + 时间(双向)冗余	2.5 kB	50:1

记忆锚点:压缩比 **7** → **20** → **50** 逐级翻倍再翻倍,恰好对应"独立 → 单向 → 双向"参考能力的增强。平均一帧约 4.8 kB、整体约 27:1。



3. H.261 编解码过程(重点)

I 帧编码

对宏块中每个 8×8 块(亮度 Y 用 16×16 宏块,4:2:0 下 Cb/Cr 用 8×8): DCT → 量化 → Zigzag 扫描 → 熵编码。和 JPEG 流程一致。

P 帧编码

对目标帧中每个宏块:

1. 用某种搜索方法(顺序/对数/分层)分配一个运动向量(H.261 中 $p=15$);
2. 用参考宏块算出差值宏块(预测误差);
3. 差值宏块的每个 8×8 块再走 DCT $\rightarrow$ 量化 $\rightarrow$ Zigzag $\rightarrow$ 熵编码。

真题陷阱 · P 帧未必用运动补偿

很多人以为"P 帧 = 运动补偿帧",这是错的。当某个宏块找不到足够好的匹配(预测误差超过可接受阈值)时,编码器会放弃预测,把这个宏块当作帧内宏块(**intra-macroblock**)直接编码——这叫非运动补偿宏块。所以"P 帧的编码一定基于运动补偿"这句话是错的:P 帧里可以混有帧内编码的宏块。

为什么参考帧必须是"已解码帧"——防止漂移 (drift)

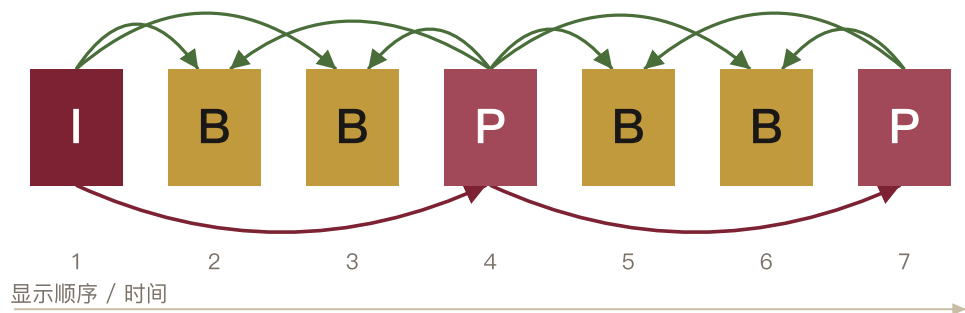
H.261 编码器内部有一个反馈环:运动估计所用的参考帧,不是原始输入帧,而是编码器自己解码重建后的帧(与 DPCM 同理)。

为什么?

解码端拿不到原始输入,它只有重建帧。如果编码端拿原始帧做参考、解码端拿重建帧做参考,两边的参考基准就不一致,误差会在一帧帧的预测链上逐帧累积、越漂越远——这就是漂移 (**drifting**)。让编码端也用"重建误差在环路里"作参考,保证两端参考帧完全相同,漂移就被消除。因此编解码器都需要一个缓冲区来保存参考帧。

4. MPEG-1 帧序列:显示顺序 $\neq$ 解码顺序

MPEG-1 在 H.261 的 I/P 基础上加入 B 帧。但 B 帧要参考它后面的 P/I 帧,而"后面的帧"必须先被解码出来——于是产生了一个关键现象:码流里帧的传输/解码顺序,和它们最终显示的顺序不一样。



— 红色(下方) = P 帧前向预测(仅参考过去) · — 绿色(上方) = B 帧双向预测(参考过去+未来)

一个 GOP:I B B P B B P 的预测方向。怎么读:箭头从**参考帧**指向**被预测的帧**。P 帧(4、7)只有一根红箭头从**过去**的 I/P 帧 伸来——单向前向预测;每个 B 帧(2、3、5、6)头顶有两根绿箭头,分别来自**过去和未来的** I/P 帧——双向预测。I 帧(1)无任何入箭头,独立自足。关键矛盾:B 帧依赖"未来"的 P 帧,可这条时间轴是**显示顺序**——未来帧尚未播出,却必须先解出来,这就逼出了下图的顺序错位。

给定一段 MPEG-1 视频的显示顺序:

$I_1 \ B_2 \ B_3 \ P_4 \ B_5 \ B_6 \ P_7$

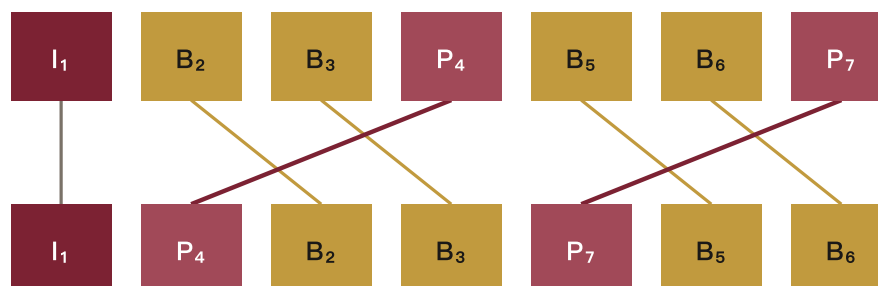
问:编码器实际的编码/解码顺序是什么?为什么?

- ① 先解 **I₁**。它独立编码,谁都不依赖,第一个就绪。
- ② 再解 **P₄**,而不是 **B₂**。因为 **P₄** 只前向参考 **I₁**,现在 **I₁** 已就绪,可以解出 **P₄**。
- ③ 回头解 **B₂**、**B₃**。它们双向参考"前面的 **I₁**"和"后面的 **P₄**"。此刻两个参考帧都已解出,**B₂**、**B₃** 才能解码。
- ④ 解 **P₇**,它前向参考 **P₄**(已就绪)。
- ⑤ 解 **B₅**、**B₆**,双向参考 **P₄** 与 **P₇**(都已就绪)。

编码/解码顺序: $I_1 \ P_4 \ B_2 \ B_3 \ P_7 \ B_5 \ B_6$

为什么:每个 **B** 帧的后向参考帧(**P₄**、**P₇**)必须先于该 **B** 帧解码。所以编码器把被参考的 **I/P** 帧提前,**B** 帧滞后——这正是显示顺序与解码顺序错位的根源。解码端解出后需重排缓冲,再按显示顺序输出。这也是 **B** 帧的两大代价之一:编解码依赖 + 额外时延,故不适合低时延应用(另一代价是双倍运动向量搜索计算量)。

显示顺序(播放)



编码/解码/传输顺序

— 红线 = P 帧被提前(它是 B 的后向参考,必须先解) · — 黄线 = B 帧被推后

同一段视频,两种顺序。怎么读:每根斜线连接的是**同一帧**在两行里的位置。P₄ 在显示里排第 4,却被**抢**到第 2 个解码(红线向左跨越),因为 B₂、B₃ 都要拿它当后向参考——参考帧必须先就绪。于是解码顺序变成 **I₁ P₄ B₂ B₃ P₇ B₅ B₆**。边界代价:解码端解出后要用缓冲区**重排**再按显示顺序输出,带来额外时延,故 B 帧不适合实时通信(H.261 就没有 B 帧)。

自测题 1

关于 MPEG-1 中的 P 帧,下列说法正确的是?

- A. P 帧独立编码,不参考任何其他帧
- B. P 帧的每个宏块都必须通过运动补偿编码
- C. P 帧可同时参考前一帧和后一帧
- D. 当某宏块找不到好的匹配时,可将其作为帧内宏块直接编码

▼ 答案与解释

答案:D

A 错:独立编码的是 I 帧。B 错:这是最常见的陷阱——P 帧未必每块都用运动补偿,匹配太差时会退化成本运动补偿宏块(帧内编码),正是 D 描述的情形。C 错:双向参考是 B 帧的特权,P 帧只做前向预测。

套路:看到"P 帧一定/必须运动补偿"直接判错。

自测题 2

某 MPEG-1 视频显示顺序为 I B B P B B P。其正确的解码顺序是?

- A. I B B P B B P
- B. I P B B P B B
- C. P I B B P B B
- D. I P P B B B B

▼ 答案与解释

答案:B(I P B B P B B)

B 帧双向参考前后的 I/P 帧,所以它的两个参考帧必须先解码。第一个 P 要参考 I(已就绪),故 P 提前到两个 B 之前;这两个 B 参考 I 和第一个 P,此时都已就绪才轮到它们。以此类推得 I P B B P B B。

核心原因:参考帧必须先于被预测帧解码,这也解释了为什么显示顺序 ≠ 解码顺序。

下一步 → 视频压缩告一段落。进入 [0019 · 基于内容的图像检索:CBIR 与 SIFT 特征](#),转向"多媒体信息检索"这条真题主线。